World Cup Prediction Model

Full Portfolio Project Plan — ML + FastAPI + React

Project overview

Build an end-to-end system that ingests historical international football data, trains an XGBoost classifier to predict match outcomes, runs Monte Carlo tournament simulations, and exposes results through a React dashboard. This project covers the full ML-to-production stack and signals the skills most valued by data, backend, and frontend recruiters.

Complete tech stack

Layer	Technology	Purpose
Data	`pandas · NumPy`	Loading, cleaning, merging datasets
Features	`scikit-learn · feature-engine`	Encoding, scaling, lag features
Model	`XGBoost`	Match outcome classifier (W/D/L)
Simulation	`Python (random / NumPy)`	Monte Carlo tournament simulation
API	`FastAPI · Uvicorn`	REST endpoints for React frontend
Frontend	`React + Vite + TypeScript`	Interactive prediction UI
Styling	`Tailwind CSS`	Utility-first responsive design
Charts	`Recharts`	Probability bars, leaderboard
Deploy	`Railway / Render`	Free-tier hosting for API + UI

Phase 1 — Data collection & feature engineering

Primary datasets

results.csv (Kaggle)

- kaggle.com — International Football Results 1872+
- 45,000+ matches · neutral venue flag included
- Filter to competitive matches only (drop friendlies)

elo_ratings.csv

- worldfootballelo.net — historical Elo ratings
- Join on (team, date) to get pre-match Elo
- Elo diff = strongest single feature in any model

Feature engineering

Feature	Type	Importance	Notes
Elo rating difference	`Numeric`	★★★★★	Most predictive single feature

Feature	Type	Importance	Notes
Recent form (last 10)	`Numeric`	★★★★■	Decay-weighted W/D/L average
Head-to-head win rate	`Numeric`	★★★■■	Historical matchup advantage
Goal difference (last 5)	`Numeric`	★★★■■	Attacking/defensive strength
Neutral venue flag	`Binary`	★★★■■	Always 1 for World Cup
FIFA ranking difference	`Numeric`	★★■■■	Slower-moving than Elo
Tournament stage weight	`Ordinal`	★★■■■	Knockout vs group stage
xG per game (Statsbomb)	`Numeric`	★★■■■	Optional — sparse coverage

Critical: avoid data leakage

- Elo ratings and form features must use data BEFORE each match's date.
- Compute rolling stats with df.shift(1) — never include the current row.
- Train on pre-2018 data, validate on 2018 & 2022 World Cups.

Phase 2 — XGBoost model training

Model architecture

Train a multi-class XGBoost classifier with three output classes: Home Win (0), Draw (1), Away Win (2). The model outputs calibrated probabilities for each class, which feed directly into the Monte Carlo simulation.

Hyperparameters

- xgb.XGBClassifier(objective='multi:softprob')
- n_estimators: 300–500 (tune with early stopping)
- max_depth: 4–6 | learning_rate: 0.05
- subsample: 0.8 | colsample_bytree: 0.8

Validation strategy

- TimeSeriesSplit — never shuffle football data
- Target: ~62–66% accuracy on competitive matches
- Evaluate per-class precision / recall
- Calibrate with CalibratedClassifierCV

Core training code

```
import xgboost as xgb
from sklearn.model_selection import TimeSeriesSplit
from sklearn.calibration import CalibratedClassifierCV
model = xgb.XGBClassifier(
objective='multi:softprob', num_class=3,
n_estimators=400, max_depth=5,
learning_rate=0.05, subsample=0.8,
eval_metric='mlogloss', early_stopping_rounds=30
)
calibrated = CalibratedClassifierCV(model, cv=3, method='isotonic')
calibrated.fit(X_train, y_train)
```

Monte Carlo simulation

- Run the full 48-team tournament 10,000 times, sampling each match from predicted probabilities.
- Track champion frequency: teams[champion] += 1 per simulation.
- Output: P(win World Cup) per team — far more impressive than single-match prediction.
- Save model with joblib.dump() for the FastAPI server to load at startup.

Phase 3 — FastAPI backend

Two endpoints are all you need

Match prediction

- POST /predict
- Body: { home_team, away_team }
- Returns: { win_prob, draw_prob, loss_prob }
- Loads pre-trained model from disk at startup

Tournament simulation

- POST /simulate
- Body: { n_simulations: 10000 }
- Returns: { team: champion_pct } sorted dict
- Runs in ~2s for 10k sims (pure NumPy)

Deployment

- Deploy to Railway.app or Render.com — both offer free tiers for Python web services.
- Add CORS middleware so your React app can call the API from a different origin.
- Include a Dockerfile in the repo: shows DevOps awareness to recruiters.

Phase 4 — React UI

Four key components

Bracket view

- Show all 8 groups and the knockout tree.
- Highlight the predicted winner of each match in real time.

Match predictor

- Two dropdowns (home / away team) + 'Predict' button.
- Call POST /predict, show W/D/L probability bars with Recharts.

Simulation runner

- 'Run 10,000 simulations' button — animated progress bar while waiting.
- Champion leaderboard sorted by win probability with animated counters.

Team comparison card

- Pick any two teams: show Elo rating, form score, head-to-head record, and flag.
- Use flagcdn.com/w40/{iso2}.png for free flag images.

Portfolio README checklist

Must-haves	Differentiators
• GIF or screenshot of the UI (non-negotiable)	• 'How it works' section — explain the ML clearly
• Architecture diagram (link to this PDF)	• 'What I'd improve' — signals engineering maturity
• Model accuracy section — show the numbers	• Reproducibility: requirements.txt + .env.example
• Live demo link (Railway free tier)	• GitHub Actions CI badge (run pytest on push)

Recruiter keyword coverage

This project hits the most searched terms across three role types:

Data / ML roles	Backend roles	Frontend roles
`pandas · scikit-learn · XGBoost feature engineering · Monte Carlo data pipeline · model evaluation`	`FastAPI · REST API design Docker · Railway deploy pydantic · async Python`	`React · TypeScript · Vite Tailwind CSS · Recharts async fetch · state management`

Ask Claude to generate the training code, FastAPI endpoints, or React components at any step.